

Multi-Agent Collaboration for Vehicular Task Offloading Using Federated Deep Reinforcement Learning

Xing Chen¹, Member, IEEE, Bohuai Xiao, Xinyu Lin, Zheyi Chen², Member, IEEE, and Geyong Min³, Member, IEEE

Abstract—Mobile Edge Computing (MEC) distributes resources such as computing, storage, and bandwidth to the side close to users, which can provide low-latency services to in-vehicle users, thus promising a more efficient and safer driving environment. However, due to the dynamic scale of vehicle and the variability of resource requirements, it is a significant challenge to quickly obtain effective task offloading in large-scale vehicle scenarios. The existing studies generally adopt the centralized decision-making method, with long decision-making time and high computational overhead, which cannot effectively achieve good offloading decisions in large-scale scenarios. To address these problems, we propose a Multi-agent Collaborative Method for vehicular task offloading using Federated Deep Reinforcement Learning called MCM-FDRL. First, each vehicle as an agent, independently makes offloading decisions based on local information. Next, the offloading decision model of each vehicle is obtained through federated reinforcement learning training. At runtime, an effective vehicle offloading plan can be gradually developed through multi-agent collaboration. Using two real-world datasets, experiments show that the MCM-FDRL has good adaptability and scalability. Moreover, compared to the state-of-the-art methods, the task's average response time of the MCM-FDRL is reduced by 9.75%-64.90%, respectively.

Index Terms—Vehicular edge computing, task offloading, deep reinforcement learning, federated learning.

I. INTRODUCTION

WITH the rapid development of the Internet-of-Things (IoT) and Artificial Intelligence (AI), the Internet-of-Vehicles (IoV) system has become increasingly diverse,

giving rise to numerous emerging applications such as intelligent voice assistants, autonomous driving, and augmented reality [1]. These applications are usually computation-intensive and latency-sensitive. However, the computing resources at vehicles are limited, which makes it challenging to accomplish these tasks under latency constraints [2]. Therefore, how to handle these tasks efficiently with limited computational resources is the key to the IoV environment.

Cloud computing [3] provides a solution for handling computation-intensive and latency-sensitive tasks with its powerful computing capabilities and flexible resource allocation. Cloud computing effectively reduces the computational pressure on users by offloading tasks to the cloud for computation and then returning the results to users. However, the distance between the cloud and users is long, and the process of offloading tasks to the cloud and returning them usually causes transmission lag and high latency [4], which makes it difficult to fit in the demands of the IoV environment.

In order to overcome the shortcomings of cloud computing, Mobile Edge Computing (MEC) [5] has been a promising way to offload tasks in IoV. As a rapidly developing computing paradigm, MEC transfers computing resources from the remote cloud to the network edges, providing users with high-performance and low-latency services. Therefore, an effective solution is to offload computation-intensive tasks from vehicles to the nearby edges for execution, utilizing the richer computing resources of edge servers to make up for the lack of computing power in vehicles. However, it is challenging to develop an effective offloading plan in real-time responsive Vehicular Edge Computing (VEC) [6]. When tasks are always uploaded to the edge, the computing resources of edges might waste and the transmission latency might rise. Moreover, when the vehicles in an area are too dense, the edge might receive too many tasks, leading to edge overloading [7]. Therefore, flexible and efficient offloading plans need to be developed for VEC to meet the decision-making requirements.

Recently, some studies have focused on solving the VEC problem using different decision-making ways. On the one hand, some studies [7], [8], [9] adopted a centralized decision-making method to obtain global real-time information from each vehicle and edge node. These methods utilized a central coordination node, such as a cloud server or a edge computing node, to process the information uploaded from each vehicle

Received 23 April 2024; revised 17 March 2025; accepted 1 April 2025. Date of publication 8 April 2025; date of current version 6 August 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62072108, in part by Fujian Province Technology and Economy Integration Service Platform under Grant 2023XRH001, in part by Fuzhou-Xiamen-Quanzhou National Independent Innovation Demonstration Zone Collaborative Innovation Platform under Grant 2022FX5, and in part by Special Funds for Promoting High-quality Development of Marine and Fishery Industries in Fujian Province under Grant FJHYF-ZH-2023-02. Recommended for acceptance by A. Abdrabou. (Corresponding authors: Zheyi Chen; Geyong Min.)

Xing Chen, Bohuai Xiao, Xinyu Lin, and Zheyi Chen are with the College of Computer and Data Science, Fuzhou University, Fuzhou 350116, China, also with the Engineering Research Center of Big Data Intelligence, Ministry of Education, Fuzhou 350002, China, and also with the Fujian Key Laboratory of Network Computing and Intelligent Information Processing (Fuzhou University), Fuzhou 350116, China (e-mail: chenxing@fzu.edu.cn; 231010016@fzu.edu.cn; linxy0826@163.com; z.chen@fzu.edu.cn).

Geyong Min is with the Department of Computer Science, University of Exeter, EX4 4QF Exeter, U.K. (e-mail: g.min@exeter.ac.uk).

Digital Object Identifier 10.1109/TMC.2025.3557898

and edge. By obtaining a global information, these methods can optimize the allocation of resources based on comprehensive data, leading to optimal decision-making results. However, when dealing with large-scale dynamic VEC scenarios, the centralized decision-making methods required massive time to search for the optimal plan, thus posing challenges in meeting timeliness requirements. On the other hand, several studies [10] adopted a distributed decision-making method for multi-vehicle task offloading. Unlike the centralized decision-making methods, these methods allowed task offloading decisions to be made independently without a central coordination node. However, the existing distributed decision-making methods usually utilized a binary offloading plan, where tasks were either offloaded or not, limiting the granularity of the decision and making the offloading inefficient.

To address the above problems, we propose a Multi-agent Collaborative Method for vehicular task offloading using Federated Deep Reinforcement Learning called MCM-FDRL. Specifically, to address the problem of excessive decision-making time in existing centralized frameworks, we employ a multi-agent distributed framework, where each agent independently makes decisions by using reinforcement learning. Through the iterative decision-making process, good coordination can be achieved among multiple agents. Moreover, to address the binary offloading problem, we adopt partial offloading, where tasks can be divided into smaller segments that can be processed simultaneously by different nodes, thus enhancing the efficiency and reducing the task response time. Furthermore, to expedite model convergence, we adopt federated learning as the training framework. The main contributions are summarized as follows.

- A multi-agent collaborative task offloading framework is designed for large-scale VEC scenarios, where each vehicle can independently make offloading decisions. Each vehicle is modeled as an agent, which independently decides the edges for task offloading based on local information, and achieves multi-vehicle collaboration and load balancing of the edges through multi-round decision making.
- A task offloading method based on federated deep reinforcement learning is proposed. First, the task decision model for each vehicle is trained using the deep q-learning network. Then, the global task decision model is obtained by aggregating the model parameters through federated learning. At runtime, each vehicle uses the task decision model to gradually form the optimal task offloading plan.
- Simulation experiments are conducted to verify the effectiveness and adaptability of the proposed MCM-FDRL in large-scale VEC scenarios. The results show that the MCM-FDRL can well handle the large-scale and dynamic problem of VEC, where the performance improvement of the MCM-FDRL is about 9.75%-64.90% compared to the state-of-the-art methods.

The rest of the paper is organized as follows. Section II reviews the related work. Section III introduces the proposed VEC system and formulated the optimization problem of VEC systems. Section IV describes the proposed MCM-FDRL in detail. Section V presents performance evaluation and analysis.

Section VI discusses some issues about applicability. Finally, Section VII concludes the paper.

II. RELATED WORK

In this section, the related work is reviewed and analyzed, including centralized decision-making methods with heuristic or control theory, centralized decision-making methods with RL, and distributed decision-making method.

Some of the existing studies on vehicle task offloading adopted centralized decision-making methods with heuristic or control theory. Dai et al. [7] considered a multi-user multi-server VEC scenario with the joint load balancing and task offloading problem. They formulated the problem as a mixed-integer nonlinear programming problem using convex optimization to find the global optimal task offloading plan. Luo et al. [8] considered the scenario of multiple vehicles competing for resources and jointly task offloading latency and cost problems. They analyzed the problem using game theory and proposed a particle swarm optimization based computation offloading method to obtain the global optimal offloading plan. Lv et al. [9] considered a resource-constrained VEC scenario and proposed an energy consumption and QoS-aware co-offloading model. They proposed a Bayesian optimization-based computation offloading method to find the optimal global offloading plan. Tiwari et al. [11] constructed a coalition graph for received tasks at the edge. They used a heuristic method to maximize the utility of global task fragments while considering latency requirements, node energy, and processing units required for task services. However, the above heuristic and control theory for centralized decision-making methods required constant sampling and iteration to obtain an approximate optimal plan in large-scale VEC scenarios, leading to long search time.

Some of the existing studies on vehicle task offloading also adopted centralized decision-making methods with RL. Jiang et al. [12] modeled the vehicle task offloading and resource allocation problem as a mixed-integer nonlinear problem, and proposed a centralized Q-Learning method to find the optimal offloading and computational resources to maximize the system utility. Zhang et al. [13] set up a vehicle offloading model with multiple edge servers. They adopted a global Deep Q-learning Network (DQN) method, while considering the selection of target servers and the confirmation of data transmission modes to obtain the optimal offloading plan. Zhang et al. [14] considered an optimal data transfer scheduling plan for a multi-vehicle environment. They used a global DQN method to achieve dynamic management and utilization of underutilized resources to obtain an optimal offloading plan. Zhan et al. [2] used a RL method to implement proximal policy optimization, which effectively extracted the potential features of the global Markov chains and made better decisions on the global task offloading plan for vehicles. However, the above RL methods for centralized decision-making required a large number of samples to learn the global policy in large-scale VEC scenarios, and the solution space of the task offloading increased accordingly, which could easily fall a local optimum and cannot make offloading decisions effectively.

TABLE I
CONTRASTIVE ANALYSIS OF DIFFERENT STUDIES ("+": INVOLVED; "-": NOT INVOLVED)

Related work	Decision-making way		Method				Offloading		No. of agents	
	Centralized	Distributed	Heuristics	Control theory	RL	FRL	Binary	Partial	Fixed	Dynamic
Dai <i>et al.</i> [7]	+	-	-	+	-	-	-	+	+	-
Luo <i>et al.</i> [8]	+	-	+	-	-	-	-	+	+	-
Lv <i>et al.</i> [9]	+	-	+	-	-	-	-	+	+	-
Tiwari <i>et al.</i> [11]	+	-	+	-	-	-	-	+	+	-
Jiang <i>et al.</i> [12]	+	-	-	-	+	-	+	-	+	-
Zhang <i>et al.</i> [13]	+	-	-	-	+	-	-	+	+	-
Zhang <i>et al.</i> [14]	+	-	-	-	+	-	-	+	-	+
Zhan <i>et al.</i> [2]	+	-	-	-	+	-	+	-	+	-
Xiao <i>et al.</i> [15]	-	+	-	+	-	-	+	-	+	-
Luo <i>et al.</i> [16]	-	+	-	+	-	-	+	-	+	-
Xu <i>et al.</i> [17]	-	+	-	-	+	-	+	-	+	-
Zhu <i>et al.</i> [18]	-	+	-	-	+	-	+	-	+	-
Fragkos <i>et al.</i> [19]	-	+	-	-	+	-	+	-	+	-
Zhang <i>et al.</i> [10]	-	+	-	-	-	+	+	-	-	+
Li <i>et al.</i> [20]	-	+	-	-	-	+	+	-	+	-
Tong <i>et al.</i> [21]	-	+	-	-	-	+	+	-	+	-
Chen <i>et al.</i> [22]	-	+	-	-	-	+	+	-	+	-
Zhou <i>et al.</i> [23]	-	+	-	-	-	+	+	-	+	-
The proposed MCM-FDRL	-	+	-	-	-	+	-	+	-	+

Several studies on the vehicle task offloading adopted a distributed decision-making method. Xiao *et al.* [15] proposed a multi-layer perceptual task offloading framework with a collaborative computational method, modeled collaborative offloading as a mixed integer nonlinear programming problem, and designed a two-layer binary intelligent firefly to minimize execution latency. Luo *et al.* [16] proposed a distributed game framework to model the multi-vehicle computational offloading problem, where binary offloading decisions are made by means of a game theory to minimize execution latency. Xu *et al.* [17] considered a VEC scenario with a fixed number of edges and vehicles, and used a distributed method with the deep spatio-temporal residual network and the asynchronous advantage actor-critic to make binary offloading decisions by each edge. Zhu *et al.* [18] considered a VEC scenario with a fixed number of edges and vehicles, and used a distributed method of multi-agent reinforcement learning with binary offloading decisions by each edge. Fragkos *et al.* [19] considered a Unmanned Aerial Vehicles (UAV)-assisted scenario with a fixed number of edges and UAVs, utilizing a decision model based on random-exploration reinforcement learning, where each UAV made binary offloading decisions. To accelerate training, some studies adopted Federated Reinforcement Learning (FRL). Zhang *et al.* [10] considered a VEC scenario with dynamic number of edges and vehicles, and used a distributed method of FRL to train a decision model at the vehicles, with binary offloading decisions made by each vehicle. Li *et al.* [20] considered a MEC scenario with a fixed number of edges and mobile devices and employed FRL to train binary-offloading decision-making models on mobile devices. Similarly, Tong *et al.* [21] used FRL to train binary-offloading decision-making models on IoT devices and optimized the federated learning process through anomaly detection. Chen *et al.* [22] handled a resource optimization scenario with a fixed number of edges and mobile devices, and they applied FRL to train binary-offloading

decision-making models on mobile devices and implemented an asynchronous parameter aggregation. Zhou *et al.* [23] considered a content delivery network with a fixed number of small base stations and user devices, and they applied FRL with personalized parameter aggregation to train the decision-making model for binary offloading. Although the above methods adopted a distributed method for decision making, they are mainly for fixed VEC scenarios and used binary offloading decisions, which reduced the efficiency of task offloading.

Table I shows the contrastive analysis of the existing studies and the proposed MCM-FDRL. Most of the existing studies adopted a centralized decision-making method based on global information, which took massive time to find an offloading plan and cannot be adapted to dynamic VEC scenarios. Several studies adopted a distributed decision-making method, but mainly considered fixed VEC scenarios and used binary offloading decisions. Different from the existing studies, we use a distributed decision-making method, where each vehicle can make partial offloading decisions based on local information, which can adapt to dynamic scenarios and get a suitable offloading plan in a faster time.

III. SYSTEM MODEL AND PROBLEM FORMULATION

In this section, the problem of multi-vehicle multi-edge cooperative computation offloading in VEC is formulated, including communication model, computation model, and problem formulation.

In the VEC scenario, vehicles continuously generate computational tasks while navigating urban roads. These tasks can be offloaded to edges connected to the vehicles through a wireless network in arbitrary proportions for processing. The dynamic of the VEC scenario arises from the constant changes in vehicle positions, as well as variations in the distribution and quantity of edge across different urban areas. In this dynamical VEC

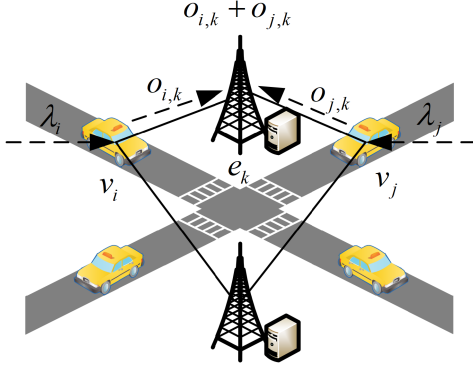


Fig. 1. Task offloading for VEC systems.

TABLE II
MAIN SYMBOLS INVOLVED IN THE PROPOSED MODEL

Symbol	Definition
N	The number of edges
M	The number of vehicles
v_i	The i -th vehicle
e_j	The j -th edge
c	The number of edges connected to a vehicle
f_{v_i}	Computing capability of v_i
f_{e_j}	Computing capability of a processor for v_i
$rt_{i,j}$	Transmission rate between v_i and e_j
λ_i	Task arrival rate at v_i
$task_i$	A task of v_i
b_i	Size of the input data of $task_i$
s_i	Required computing resources of $task_i$
O_i	Task offloading plan of v_i
$o_{i,0}$	Task volume executed on v_i in unit time
$o_{i,j}$	Task volume scheduled from v_i to e_j in unit time
w_{v_i}	Load at v_i in unit time
w_{e_j}	Load at e_j in unit time
ρ_{v_i}	Load rate at v_i
ρ_{e_j}	Load rate at e_j
t_i	Average response time of $task_i$
T	Average task response time among all vehicles

scenario, vehicles need to make rapidly decisions on task offloading to minimize overall response time. Therefore, we consider the problem of cooperative computation between multiple vehicles and edges in a dynamic VEC scenario. As shown in Fig. 1, the VEC scenario consists of M vehicles and N edges, and each edge is located on wireless access points. The tasks generated by the vehicles can be partitioned in arbitrary proportions and offloaded to any of the connected edge for processing through the wireless network. Each vehicle makes offloading decisions independently based on the local information, and multiple vehicles obtain the final offloading plan through collaboration. The major symbols used in this paper are defined in Table II.

A. Communication Model

We denote M vehicles by the set $\mathcal{V} = \{v_1, v_2, \dots, v_M\}$ and N edges by the set $\mathcal{E} = \{e_1, e_2, \dots, e_N\}$, where v_i denotes the

i th vehicle and e_j denotes the j th edge. Vehicles and edges are connected through the Vehicle-to-Infrastructure (V2I) links.

We denote the transmission rate between M vehicles and N edges by the $M \times N$ two-dimensional matrix RT :

$$RT = \begin{bmatrix} rt_{1,1} & \dots & rt_{1,N} \\ \vdots & \ddots & \vdots \\ rt_{M,1} & \dots & rt_{M,N} \end{bmatrix}, \quad (1)$$

where $rt_{i,j}$ denotes the transmission rate between vehicle v_i and edge e_j , defined as:

$$rt_{i,j} = B \log_2 \left(1 + \frac{P(d_{i,j})^{-\alpha}}{\sigma^2} \right), \quad (2)$$

where B denotes the bandwidth of the V2I link, P denotes the transmission power of the vehicle, σ^2 denotes the Gaussian white noise power of V2I link, $d_{i,j}$ denotes the distance between vehicle v_i and edge e_j , α denotes the path loss exponent, and $(d_{i,j})^{-\alpha}$ denotes the power gain of V2I link.

Each vehicle connects to several edges with the shortest distance (i.e., the best transmission rate) and makes offloading decisions independently. Due to the mobility of vehicles, the edges connected to vehicles are changeable at different times. The number of edges connected to each vehicle is denoted as c , and the set of edges currently connected to the vehicle v_i is denoted as $\mathcal{E}_i = \{e_1^i, e_2^i, \dots, e_c^i\}$.

B. Computation Model

We use $\mathcal{F}_v = \{f_{v_1}, f_{v_2}, \dots, f_{v_M}\}$ to denote the computing power of M vehicles, where f_{v_i} denotes the computing power of vehicle v_i . We denote the task arrival rate of M vehicles by $\lambda = \{\lambda_1, \lambda_2, \dots, \lambda_M\}$, where λ_i denotes the task arrival rate of vehicle v_i , which is the average task volume that vehicle v_i arrives at per unit time. The task generation process for vehicle v_i follows a Poisson process with parameter λ_i [24]. The tasks of a vehicle can be offloaded to c connected edges for execution, as mentioned in Section III-A. The task of vehicle v_i is defined as a tuple $task_i = (b_i, s_i)$, where b_i denotes the size of data input to the task (in bits), and s_i denotes the number of computing resources required for each bit of data for the task (in cycles/bits).

We denote the task offloading plan for M vehicles by a two-dimensional matrix O :

$$O = \begin{bmatrix} O_1 \\ \vdots \\ O_M \end{bmatrix} = \begin{bmatrix} o_{1,0} & o_{1,1} & \dots & o_{1,N} \\ \vdots & \vdots & \ddots & \vdots \\ o_{M,0} & o_{M,1} & \dots & o_{M,N} \end{bmatrix}, \quad (3)$$

when $j = 0$, $o_{i,0}$ denotes the volume executed locally by the task of vehicle v_i in unit time, when $j \neq 0$, $o_{i,j}$ denotes the volume that vehicle v_i offloads on edge e_j in unit time. Specifically, if vehicle v_i and edge e_j are not connected, $o_{i,j} = 0$. In addition, each edge can only schedule a vehicle's task back to the original vehicle but not to other vehicles.

We define $\mathcal{W}_v = \{w_{v_1}, w_{v_2}, \dots, w_{v_M}\}$ as the actual task load of M vehicles per unit time, where w_{v_i} denotes the actual task volume of vehicle v_i per unit time, including the task

volume executed at local (i.e., $w_{v_i} = o_{i,0}$). Similarly, we define the actual task load of N edges per unit time as $\mathcal{W}_e = \{w_{e_1}, w_{e_2}, \dots, w_{e_N}\}$, where w_{e_j} denotes the actual task volume of edge e_j per unit time, including the task volume offloaded from all vehicles (i.e., $w_{e_j} = \sum_{i=1}^M o_{i,j}$).

We use the M/M/1 queuing model [25] to model the task processing of vehicles. We denote the service rate of M vehicles by $\mu_v = \{\mu_{v_1}, \mu_{v_2}, \dots, \mu_{v_M}\}$, where μ_{v_i} denotes the service rate of vehicle v_i (i.e., the task volume that vehicle v_i can process per unit time). We denote the load rate of M vehicles by $\rho_v = \{\rho_{v_1}, \rho_{v_2}, \dots, \rho_{v_M}\}$, where ρ_{v_i} denotes the load rate of vehicle v_i . The service rate μ_{v_i} and the load rate ρ_{v_i} of vehicle v_i can be calculated as [26]:

$$\mu_{v_i} = \frac{f_{v_i}}{b_i s_i}, \quad (4)$$

$$\rho_{v_i} = \frac{w_{v_i}}{\mu_{v_i}}. \quad (5)$$

Based on the M/M/1 queuing model, the task average execution time t_{v_i} (i.e., the execution time per unit task on vehicle v_i), is defined as:

$$t_{v_i} = \frac{1}{\mu_{v_i} - w_{v_i}}. \quad (6)$$

Similarly, we use the M/M/n [25] queuing model to model the task processing of edges, where n denotes the number of homogeneous processors. We denote the computing power of N edges by the set $\mathcal{F}_e = \{f_{e_1}, f_{e_2}, \dots, f_{e_N}\}$, where f_{e_j} denotes the computing power of one processor in edge e_j . We use $\mu_e = \{\mu_{e_1}, \mu_{e_2}, \dots, \mu_{e_N}\}$ to denote the service rate of edges, where μ_{e_j} denotes the service rate of the edge e_j (i.e., the task volume that the edge e_j can process per unit of time). We denote the load rate of N edges by $\rho_e = \{\rho_{e_1}, \rho_{e_2}, \dots, \rho_{e_N}\}$, where ρ_{e_j} denotes the load rate of edge e_j . The service rate μ_{e_j} and load rate ρ_{e_j} of edge e_j can be calculated as:

$$\mu_{e_j} = \frac{f_{e_j}}{bs}, \quad (7)$$

$$\rho_{e_j} = \frac{w_{e_j}}{n \cdot \mu_{e_j}}, \quad (8)$$

where bs denotes the expected value of the number of computing resources required for all vehicle tasks, calculated based on historical data [26].

Based on the M/M/n queuing model, the task average execution time t_{e_j} on edge e_j (i.e., the execution time of a unit task), is defined as:

$$t_{e_j} = \frac{C(n, \rho_{e_j})}{n \cdot \mu_{e_j} - w_{e_j}} + \frac{1}{\mu_{e_j}}, \quad (9)$$

where $C(n, \rho)$ is the Erlang-C formula, which denotes the probability that tasks arriving within a unit of time cannot be processed promptly and must be queued, denoted as:

$$C(n, \rho) = \frac{\frac{(n\rho)^n}{n!} \cdot \frac{1}{1-\rho}}{\sum_{k=0}^{n-1} \frac{(n\rho)^k}{k!} + \frac{(n\rho)^n}{n!} \cdot \frac{1}{1-\rho}}. \quad (10)$$

The tasks of a vehicle are often distributed to be executed locally and on the nearby c adjacent edges. Furthermore, the various task components are executed in parallel. Therefore, the average response time of each vehicle's arrival task is the maximum of the response time of its component tasks. Therefore, we define the average response time t_i for the task on vehicle v_i as:

$$t_i = \max(o_{i,0} \cdot t_{v_i}, o_{i,1} \cdot t_{v_i, e_1}, \dots, o_{i,N} \cdot t_{v_i, e_N}), \quad (11)$$

where t_{v_i, e_N} denotes the average response time of tasks from the tasks of vehicle v_i that are scheduled and executed on the edge e_j within a unit of time. So, t_{v_i, e_N} is defined as:

$$t_{v_i, e_N} = t_{e_j} + \frac{b_i}{rt_{i,j}}. \quad (12)$$

C. Problem Formulation

To further evaluate the task offloading effect of M vehicles, we define T to denote the average response time of arrival tasks on M vehicles:

$$T = \frac{1}{M} \sum_{i=1}^M t_i. \quad (13)$$

Finally, we define the objective function as:

$$\begin{aligned} P_1 : & \min_O T \\ \text{s.t. } & C1 : o_{i,j} \geq 0, \forall v_i \in V, j \in \{0, 1, \dots, N\} \\ & C2 : \sum_{j=0}^N o_{i,j} = \lambda_i, \forall v_i \in V \\ & C3 : 0 \leq \rho_{v_i} \leq 1, \forall v_i \in V \\ & C4 : 0 \leq \rho_{e_j} \leq 1, \forall e_j \in E \end{aligned} \quad (14)$$

where $C1$ and $C2$ denote the constraints of task offloading volume for each vehicle, and $C3$ and $C4$ denote the load rate constraints for each vehicle and each edge, respectively.

After using different task offloading plan O for task offloading, the average response time T for tasks on M vehicles also vary. In order to achieve a better task offloading effect in the VEC system, our goal is to minimize the average response time T of the task on M vehicles as much as possible.

IV. TASK OFFLOADING OF LARGE-SCALE VEHICLES WITH THE PROPOSED MCM-FDRL

Fig. 2 shows the framework of the proposed MCM-FDRL. First, we model each vehicle as an agent and design a Deep Reinforcement Learning (DRL) based task offloading method (Section IV-A). Next, we design a federated learning based training framework (Section IV-B). Then, we propose a runtime decision-making algorithm (Section IV-C). Finally, we analyze the complexity of the MCM-FDRL (Section IV-D).

More specifically, Algorithm 1 gives the details. First, update the vehicle's local information, which is described in Section IV-A (line 4). These local information are continuously

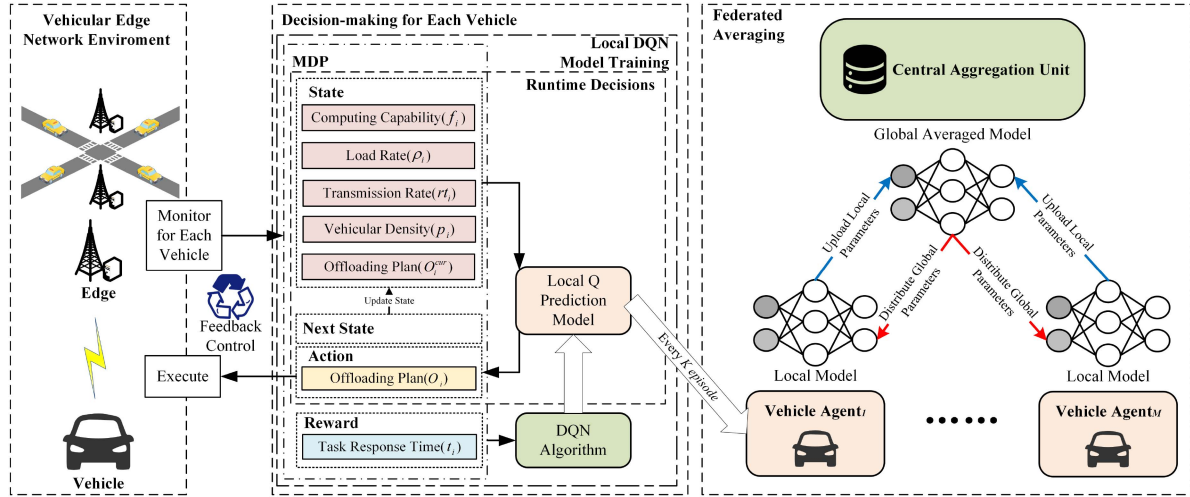


Fig. 2. The framework of MCM-FDRL for task offloading in the VEC systems.

changing as other vehicles also offload. Therefore, local information should be re-obtained before each round of decision-making. Next, by using local information as the input of a runtime decision algorithm (Algorithm 4), we can get v_i 's next offloading operation (line 5). If the return of Algorithm 4 is null, v_i has found a suitable plan locally, no further adjustment is needed (lines 6-8). Otherwise, it executes the computed offloading operation by Algorithm 4 (line 9). Finally, loop through the above process and continuously schedule the offloading of tasks for the vehicle and other vehicles. Algorithm 1 includes the following three procedures:

Procedure 1 (Section IV-A): We model each vehicle as an agent, and each vehicle can make independent decisions based on local information collected in the runtime environment, find a suitable task offloading plan to minimize its task response time t_i , and thus minimize the average response time T of the system. By designing state space, action space, state-transition function, and reward function, we model vehicles' dynamic task offloading problem as a Markov Decision Process (MDP), and the local model is trained using local information in the runtime environment.

Procedure 2 (Section IV-B): We use the federated deep reinforcement learning algorithms to train a Q-value prediction model for vehicle task offloading operations. We send the trained local model parameters to the central aggregation unit and then train the global model by federated aggregation. In this process, the local information of vehicle is inputted, and the trained model can accurately evaluate the Q-value of the vehicle's task offloading operation in different scenarios and different operating environments.

Procedure 3 (Section IV-C): In the runtime, each vehicle uses a trained federal model for Q-value prediction of vehicle task offloading operations to predict the Q-values of different task offloading operations based on the local information in the runtime environment and selects the vehicle task offloading operation with the highest Q-value. Next, through multi-vehicle

Algorithm 1: Distributed Decision-Making of Task Offloading for Each Vehicle.

```

1 Input:  $\lambda_i$  and  $f_{v_i}$ .
2 Initialize  $O_i^{cur} = (\lambda_i, 0, \dots, 0)$ .
3 while true do
4   Update local information;
5   Get the next adjustment operation of task
   offloading  $a_i = \text{Algorithm 4}(\text{local information})$ ;
6   if  $a_i$  is null then
7     Continue;
8   else
9     Execute the adjustment operation of task
     offloading  $a_i$  at  $v_i$ ;
10  end
11 end

```

collaboration, the algorithm can gradually find an effective large-scale vehicle task offloading plan.

A. DRL for Vehicle Task Offloading Decisions

We model task offloading process as a MDP and use the DQN [27] to train the task offloading decision model.

Based on the local information collected from the runtime environment, each vehicle independently schedules tasks between vehicles and adjacent edges. The local information collected by a vehicle (e.g., v_i) includes the computing power at v_i and its adjacent edges (denoted by f_i), the load rate at v_i and its adjacent edges (denoted by ρ_i), the transmission rate between v_i and its adjacent edges (denoted by rt_i), the density of vehicles at its adjacent edges (denoted by p_i), and the current local task offloading plan at e_i (denoted by O_i^{cur}) with the corresponding average response latency of the tasks (denoted by t_i), as shown in Table III. Specifically, v_i is connected to c adjacent edges, denoted by $\mathcal{E}_i = \{e_1^i, e_2^i, \dots, e_c^i\}$, where e_j^i ($1 \leq j \leq c$) denotes

TABLE III
LOCAL INFORMATION OF VEHICLES IN THE RUNTIME ENVIRONMENT

f_i	ρ_i	rt_i	p_i	O_i^{cur}
$f_0^i, f_1^i, f_2^i, \dots, f_c^i$	$\rho_0^i, \rho_1^i, \rho_2^i, \dots, \rho_c^i$	$rt_1^i, rt_2^i, \dots, rt_c^i$	$p_1^i, p_2^i, \dots, p_c^i$	$o_0^i, o_1^i, o_2^i, \dots, o_c^i$

the j th edge that is connected to v_i . In particular, the tasks at a vehicle can only be executed at the local or adjacent edges. Moreover, $f_i = (f_0^i, f_1^i, f_2^i, \dots, f_c^i)$, where f_0^i denotes the computing power at v_i , and f_j^i ($1 \leq j \leq c$) denotes the computing power at e_j^i . In addition, $\rho_i = (\rho_0^i, \rho_1^i, \rho_2^i, \dots, \rho_c^i)$, where ρ_0^i denotes the load rate at v_i , and ρ_j^i ($1 \leq j \leq c$) denotes the load rate at e_j^i . Besides, $rt_i = (rt_1^i, rt_2^i, \dots, rt_c^i)$, where rt_j^i ($1 \leq j \leq c$) denotes the transmission rate between v_i and e_j^i . Furthermore, $p_i = (p_1^i, p_2^i, \dots, p_c^i)$, where p_j^i ($1 \leq j \leq c$) denotes the density of vehicle of edge e_j^i . Thus, $O_i^{cur} = (o_0^i, o_1^i, o_2^i, \dots, o_c^i)$, where o_0^i denotes the task volume in the tasks that are executed at v_i , and o_j^i ($1 \leq j \leq c$) denotes the task volume in the tasks at v_i that is scheduled to e_j^i and executed at e_j^i . After scheduling tasks based on O_i^{cur} , the corresponding t_i can be calculated.

For task offloading of vehicles, we use the DQN to train the task offloading decision model. As shown in Fig. 2, the DQN agent v_i first receives the state s in the runtime VEC environment and chooses the action a through the ϵ -greedy strategy. Next, it obtains the corresponding reward r , and steps to the next state s' . The experience (s, a, r, s') is stored in the replay buffer with the predefined capacity at each step. Let $|x| = |r + \gamma \max_{a'} Q(s', a'; \theta') - Q(s, a; \theta)|$ denote the error between the Q-value prediction model and the estimated true Q-value, and then the Huber loss function [28] is defined as:

$$huber_loss = \begin{cases} 0.5x^2 & |x| \leq \delta \\ 0.5\delta^2 + \delta(|x| - \delta) & |x| > \delta \end{cases}, \quad (15)$$

where γ is the discount factor, θ is the weights of DNNs, $Q(s, a; \theta)$ is the current Q-value, $\max_{a'} Q(s', a'; \theta')$ is the maximum Q-value when choosing a' at s' , and δ is a pre-set threshold.

The DRL problem can commonly be formulated as the MDP [29], which aims to maximize long-term rewards. Specifically, an MDP can be described as a 4-tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{ST}, \mathcal{R} \rangle$, where $\mathcal{S}, \mathcal{A}, \mathcal{ST}$, and $\mathcal{R}$ denote the state space, action space, state transition function, and reward function, respectively. According to the Section III definitions of cooperative task offloading problem in VEC, they are defined as:

State space: The state space of v_i is denoted as $\mathcal{S}_i$, where $s_j^i \in \mathcal{S}_i$ indicates a potential state. Specifically, s_j^i is defined as a 5-tuple $\langle f_i, \rho_i, rt_i, p_i, O_i^{cur} \rangle$, where the detailed descriptions are given in Table III. If O_i^{cur} will cause the load on the vehicle or edge to exceed the adjustment range, then consider s_j^i is in an illegal state, resulting in O_i^{cur} is also illegal.

Action space: The action space of v_i is defined as $\mathcal{A}_i = \{add_1^i, dec_1^i, add_2^i, dec_2^i, \dots, add_c^i, dec_c^i\}$, where an action (i.e., task offloading operation) is to add or decrease the task volume in the tasks at v_i that is scheduled to an adjacent edge. For instance, add_j^i adds the task volume in the tasks at v_i that is scheduled to e_j^i . The task volume to be added or decreased by an adjustment operation is a fixed value of ϑ .

State-transition function: The state-transition function is defined as $\mathcal{ST}(s, a)$, whose return is the next state after executing a . For example, when executing add_1^i at $s_0^i = \langle (f_0^i, f_1^i, f_2^i, \dots, f_c^i), (\frac{w_{v_i}}{\mu_{v_i}}, \frac{w_{e_1^i}}{n \cdot \mu_{e_1^i}}, \frac{w_{e_2^i}}{n \cdot \mu_{e_2^i}}, \dots, \frac{w_{e_c^i}}{n \cdot \mu_{e_c^i}}), (rt_1^i, rt_2^i, \dots, rt_c^i), (p_1^i, p_2^i, \dots, p_c^i), (o_0^i, o_1^i, o_2^i, \dots, o_c^i) \rangle$, the next state $s_1^i = \langle (f_0^i, f_1^i, f_2^i, \dots, f_c^i), (\frac{w_{v_i}-\vartheta}{\mu_{v_i}}, \frac{w_{e_1^i}+\vartheta}{n \cdot \mu_{e_1^i}}, \frac{w_{e_2^i}}{n \cdot \mu_{e_2^i}}, \dots, \frac{w_{e_c^i}}{n \cdot \mu_{e_c^i}}), (rt_1^i, rt_2^i, \dots, rt_c^i), (p_1^i, p_2^i, \dots, p_c^i), (o_0^i, o_1^i, o_2^i, \dots, o_c^i) \rangle$ can be observed, where the load at v_i is decreased by ϑ and the load at e_1^i is added by ϑ .

Reward function: The reward function is used to guide the DQN agent to learn the optimal deployment plan for minimizing the average response time of the whole system, so it is defined as:

$$r_i = \mathcal{R}(s_i, a_i) = t_i - t'_i, \quad (16)$$

where r_i denotes the reward after taking a_j at s_j , t_i and t'_i denote the response time of the vehicle v_i task offloading plan before and after taking a_i , respectively. Thus, r_i is a negative value. To maximize the cumulative rewards, the response time should be minimized.

Finally, we introduce the density of vehicles at the edges in the MCM-FDRL. In traditional methods, a centralized decision-making method trains the Q-value prediction model, allowing vehicles to perceive global information and avoid high edge load. In distributed decision-making, vehicles make independent decisions, resulting in excessive load on edges with concentrated vehicles and increased latency. Therefore, we propose the density of vehicles at the edges, representing the edge's future load level. Suppose the density of vehicles at the edges is higher. In that case, it indicates that the computing resources of all edges within the edge area make it more challenging to process the tasks of vehicles in the area, and vehicles in the area need to avoid offloading to the edge. The critical steps of calculating the density of vehicles at the edges are shown in Algorithm 2. Consider dividing a circular area with edge e as the center and D as the radius. First, initialize the number of edges n_{e_j} and vehicles m_{e_j} within the circular area (line 3). Next, count the number of vehicles (lines 4~9) and edges (lines 10~15) within the area. Finally, calculate the density of vehicles ρ_{e_j} at the edge e_j by (17) (line 16):

$$\rho_{e_j} = \frac{m_{e_j}}{n_{e_j}}. \quad (17)$$

Fig. 3 shows a scenario centered on edge e at latitude and longitude (121.4511, 31.2289) at 23:00 in the datasets (introduced in Section V). In this scenario, the number of edges n_e is 5, and the number of vehicles m_e is 17. Therefore, the density of vehicles ρ_e at the edge e is 3.4.

B. Q-Value Prediction Model Training Based on Federated Learning

The proposed MCM-FDRL allows agents to train their local models in a distributed method and then send their local models to the central aggregation unit to build a global model. Unlike centralized decision-making, the MCM-FDRL only needs to let

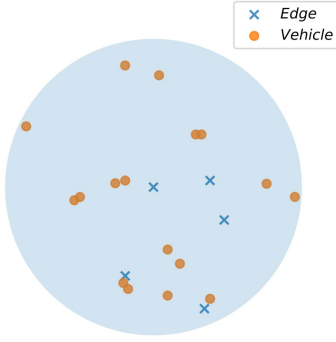


Fig. 3. Example of vehicle density calculation at the edge.

Algorithm 2: Calculate Vehicle Density for Each Edge.

```

1 Input: The locations of vehicles and edges in the
   current runtime environment.
2 Output: The density of vehicles at the edge  $e_j$   $\rho_{e_j}$ .
3 Initialize The number of edges  $n_{e_j} = 0$  and the
   number of vehicles  $m_{e_j} = 0$  in the circular region
   centered on edge  $e_j$  with radius  $D$ .
4 for each  $v_i \in V$  do
5   Calculate the distance  $d_{i,j}$  between  $v_i$  and  $e_j$ 
   according to their locations;
6   if  $d_{i,j} \leq D$  then
7     Update  $m_{e_j} = m_{e_j} + 1$ ;
8   end
9 end
10 for each  $e_i \in E$  do
11   Calculate the distance  $d_{i,j}$  between  $e_i$  and  $e_j$ 
   according to their locations;
12   if  $d_{i,j} \leq D$  then
13     Update  $n_{e_j} = n_{e_j} + 1$ ;
14   end
15 end
16 Calculate  $\rho_{e_j}$  by Eq. (17);
17 return  $\rho_{e_j}$ ;

```

each agent upload the model parameters, significantly reducing communication costs. In the MCM-FDRL, the local models of all agents have the same structure of neural networks, and they are aggregated to form a global model through federated averaging. This training process consists of two parts as follows:

- 1) *Distributed training*: Each agent customizes the standard DQN to train the local model. After the local training, the model parameters are extracted from each agent and sent to the central aggregation unit.
- 2) *Federated aggregation*: When the central unit receives the model parameters of each agent, it aggregates these parameters to generate a global model. Then, it distributes the global model parameters to all agents to update their local models. We perform model aggregation using the FedAvg [30]:

$$\theta_g = \frac{1}{M} \sum_{i=1}^M \theta_i, \quad (18)$$

where θ_g and θ_i are the network parameters of v_i 's global model and local models, respectively.

Based on the above definition, we use the MCM-FDRL to train a federated model for evaluating the Q-values of all vehicle task offloading operations. The main steps are outlined in Algorithm 3. DQN parameters are randomly initialized (line 3), and training is performed on the datasets (line 5). During every runtime interaction (line 7), an action a_i is randomly chosen with the probability of ϵ [27]. Rewards are calculated, and new states are generated (lines 10-11). Experiences are stored in the replay buffer (line 12). Network parameters are updated using mini-batch and Huber loss according to (15) (lines 15-16). Local network parameters are uploaded to the central unit periodically (line 19). The central unit fits the global Q-network using FedAvg (line 20). Network parameters are distributed to vehicle agents for updating local networks (lines 20-22). Finally, the algorithm continues training until convergence.

C. Runtime Decisions for Vehicle Task Offloading Operations

In this section, we propose a new runtime decision algorithm to find a task offloading plan for each vehicle. Algorithm 4 describes the main steps of the decision-making process. First, the $Q_{predModel}$ is called to calculate the Q-values of all actions (i.e., task offloading operations) and store them into the set Q_{values} (line 11). If the local task offloading plan O_i^{next} after selecting action a_i is illegal, then the action is considered illegal, and the corresponding Q-values will be indexed by $-\infty$ (lines 14~16). Next, a predefined threshold τ is used to determine whether the objective task offloading plan is found. This is because some errors exist in Q-value prediction for task offloading operations. If the Q-values of all valid adjustment operations are less than τ , the objective task offloading plan is considered to be found. Thus, there is no need to execute adjustment operations anymore (lines 18~19). Otherwise, the adjustment operation with the maximum Q-value will be selected (lines 20~22). Finally, return to the task offloading operation (line 24).

D. Complexity Analysis of MCM-FDRL

The size of the state space and action space of a single vehicle agent are $|s|$ and $|a|$, respectively, and each agent has the same neural network structure. When calculating the time complexity of MCM-FDRL, the following two steps need to be considered:

- 1) *Action selection*: Let the number of hidden layers in the DQN neural network be L , and the number of neurons in the i th ($i \in [0, L)$) layer be U_i , then the complexity of the layer i is $O(U_{i-1}U_i + U_iU_{i+1})$. Therefore, for a single agent, the time complexity of neural network inference is $C_t^A = O(|s|U_0 + \sum_{i=0}^{L-2} (U_{i-1}U_i + U_iU_{i+1}) + U_{L-1}|a|)$.
- 2) *Reward calculation*: For a single agent, the time complexity of calculating reward values is $C_t^R = O(|s|)$.

Algorithm 3: Federated DQN for Training Q-Value Prediction Models for Vehicle Task Offloading Operations.

```

1 Input:  $f_i, \rho_i, rt_i, p_i$  and  $O_i^{cur}$  of each vehicle.
2 Output: The Q-value prediction federated model of vehicle task offloading operations.
3 Initialize the replay buffer  $B_i$  of each vehicle agent  $v_i$  with the capacity  $M_b$ , the action-value function  $Q$  with random weight  $\theta_i = \theta_g$  in vehicle agent  $v_i$ , and the target action-value function  $\hat{Q}$  with weight  $\theta'_i = \theta_i$  in vehicle agent  $v_i$ .
4 for each episode do
5   for each data in datasets do
6     Call Algorithm 2 to get vehicle density of edges;
7     Initialize  $s_i^{cur} = \langle f_i, \rho_i, rt_i, p_i, O_i^{cur} \rangle$ ;
8     for each step do
9       Select an action  $a_i$  with the probability of  $\epsilon$  or with  $\arg \max_{a_i} Q(s_i^{cur}, a_i)$ ;
10      Execute  $a$  and receive reward  $r_i = \mathcal{R}(s_i^{cur}, a_i)$ ;
11      Receive the next state  $s'_i = \mathcal{ST}(s_i^{cur}, a_i)$ ;
12      Store  $(s_i^{cur}, a_i, r_i, s'_i)$  into  $B_i$ ;
13      Update  $s_i^{cur} = s'_i$ ;
14    end
15    Draw  $m$  samples from  $B_i$  by the mini-batch and calculate  $Q_i^{target} = \begin{cases} r_i & \forall a'_i(a'_i \text{ is invalid}); \\ r_i + \gamma \max \hat{Q}(s'_i, a'_i; \theta'_i) & \text{otherwise} \end{cases}$ ;
16    Update  $\theta_i$  by the huber loss function;
17  end
18  if episode mod  $K$  is 0 then
19    Upload the model parameter  $\theta_i$  to the central aggregation unit;
20    Download the global model parameter  $\theta_g$  from the central aggregation unit;
21    Update  $\theta_i = \theta_g$ ;
22    Update  $\theta'_i = \theta_i$ ;
23  end
24 end

```

By combining the above two steps, the time complexity of training a single agent at time step t is $C_t = C_t^A + C_t^R$. Therefore, the time complexity of training an agent is $O(T \times C_t)$, where T is the total time step required for training. For the federated reinforcement learning training process, the time complexity of MCM-FDRL is $O(M \times T \times C_t)$.

V. PERFORMANCE EVALUATION

In this section, the simulation settings are introduced, and the proposed MCM-FDRL is evaluated considering the following Research Questions (RQs):

- **RQ 1:** How much is the performance improvement of the MCM-FDRL compared to other baselines? (Section V-B)
- **RQ 2:** How will the number of vehicles and edges at different scales affect the MCM-FDRL? (Section V-C)

Algorithm 4: Runtime Decision-Making for Vehicle Task Offloading Operations.

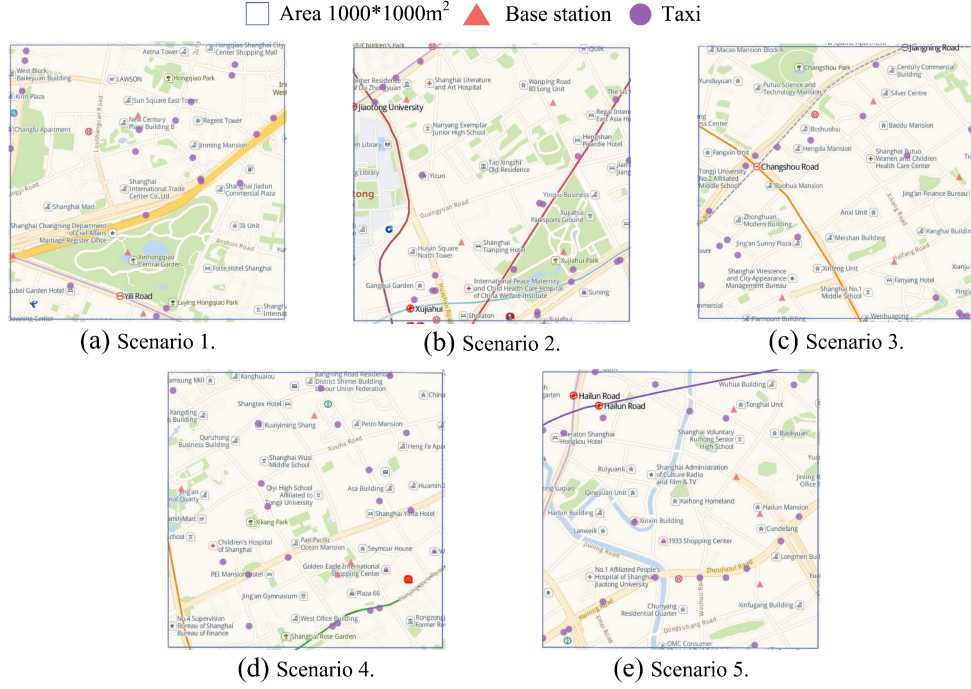
```

1 Input: The runtime local information of  $v_i$  including  $f_i, \rho_i, rt_i, p_i$  and  $O_i^{cur}$ .
2 Output: The next action  $a_i$  (offloading adjustment operation) at  $v_i$ .
3 Declaration:  $\mathcal{A}_i$ : action space containing all possible offloading adjustment operation, where  $a_i \in \mathcal{A}_i$ .
4  $Q_{predModel}$ : the Q-value prediction federated model.
5  $Q_{values}$ : the Q-values of all actions.
6  $getNextPlan(O_i^{cur}, a)$ : return the task offloading plan after selecting action  $a$ .
7  $a_{list}$ : the set of action  $a$  with the maximum Q-value.
8 Initialize the density of vehicles at the edges by using Algorithm 2.
9 Update current state  $s_i^{cur} = \langle f_i, \rho_i, rt_i, p_i, O_i^{cur} \rangle$ ;
10  $Q_{values} = Q_{predModel}(s_i^{cur}; \theta_g)$ ;
11 for each  $a_i \in \mathcal{A}_i$  do
12    $O_i^{next} = getNextPlan(O_i^{cur}, a_i)$ ;
13   if  $O_i^{next}$  is invalid then
14      $Q_{values}[a_i] = -\infty$ ;
15   end
16 end
17 if  $\forall Q_{value} \in Q_{values} (Q_{value} < \tau)$  then
18    $a_i = \text{null}$ ;
19 else
20   Record the action with the maximum Q-value:  $a_{list} = \arg \max_a (Q_{values})$ ;
21   Randomly select an action from  $a_{list}$ :  $a_i = a_{list}.getActionRandom()$ ;
22 end
23 return  $a_i$ ;

```

- **RQ 3:** How is the convergence performance of the MCM-FDRL? (Section V-D)
- **RQ 4:** Does the MCM-FDRL of using the density of vehicles at the edges as states help optimize the average response time? (Section V-E)

For **RQ 1**, the proposed MCM-FDRL outperforms the PSO-GA, C-DQN, Random, Greedy, B-FDQN, and B-FDDQN average by 9.75%, 52.06%, 21.11%, 38.53%, 64.90%, and 64.90% in terms of the average response time for different scenarios, respectively. Moreover, the proposed MCM-FDRL can obtain vehicle task offloading plans more quickly, which reduces the execution time by more than 99% compared to the PSO-GA and Random, and is of the same order of magnitude as C-DQN, Greedy, B-FDQN, and B-FDDQN. For **RQ 2**, in large-scale experiments, the proposed MCM-FDRL shows a better average response time than other methods. For **RQ 3**, the MCM-FDRL can achieve quick convergence and keep stable. For **RQ 4**, when using algorithm 2, the average response time obtained by the MCM-FDRL decreases by 5.04%; meanwhile, the execution time decreases by 30.46%.

Fig. 4. Example of $1000 \times 1000 \text{ m}^2$ areas in VEC system.TABLE IV
SETTINGS OF SIMULATION AREAS

Area	Area size (m^2)	No. of edges	No. of vehicles
1	1000×1000	4	18~22
2	1500×1500	9	43~47
3	2000×2000	16	78~82

A. Simulation Settings

The simulation experiments are conducted based on the datasets of Base Station (BS) distribution in Shanghai¹ [31], [32] [33] and taxi distribution in Shanghai² with longitude and latitude information. To simulate the diversity of scenario, we consider 15 different scenarios varying in the area size, number of edges, and number of vehicles. In each scenario, the edges and vehicles are randomly distributed, and each vehicle is connected to three edges (i.e., $c = 3$). The specific settings are given in Table IV. As shown in Fig. 4, for the area of $1000 \times 1000 \text{ m}^2$, we randomly select 5 scenarios, where the number of edges and vehicles are 4 and 18-22, respectively. For the area of $1500 \times 1500 \text{ m}^2$, we randomly select 5 scenarios, where the number of edges and vehicles are 9 and 43-47, respectively. For the area of $2000 \times 2000 \text{ m}^2$, we randomly select 5 scenarios, where the number of edges and vehicles are 16 and 78-82, respectively. The CPU-cycle frequencies of vehicles f_{v_i} are 1~2 GHz [34]. Each edge equips 4 homogeneous processors [26], and the CPU-cycle frequency of each processor f_{e_j} is 2.5~5 GHz [10]. The task arrival pattern of each vehicle

follows the Poisson process with parameter λ_i is 0.7 [35]. The size of input data of tasks b_i meets a random distribution within 10~20 Mbit [34], and the required computing power for processing data s_i is 100 cycle/bit [36]. The transmission power of vehicles P is 30 dBm, the path loss exponent α is 4, the wireless channel bandwidth B is 1MHz, and the Gaussian white noise power σ^2 is -100 dBm [36]. The task volume to be added or decreased at e_i by an adjustment operation ϑ_i is set to $4\% \times \lambda_i$ [37]. The radius for calculating the density of vehicles at the edges D is 500 m.

Next, the local information is constructed and collected separately for each vehicle in the runtime environment by simulating different scenarios. The datasets for each vehicle contain f_i , ρ_i , rt_i , p_i and O_i^{cur} (defined in Table III). The proposed MCM-FDRL is implemented based on TensorFlow 2.13.0 [38]. In the MCM-FDRL, the fully connected DNNs are used that consist of one input layer, two hidden layers, and one output layer, where these two hidden layers both have 128 hidden neurons, and their activation function is Rectified Linear Unit (ReLU) [39]. The memory capacity of experience replay buffer M_b , the training batch size m , the discount factor γ , and the learning rate of the Adam optimizer [40] are set to 100000, 32, 0.99, and 0.0005, respectively. The parameter δ of Huber loss is 1. Moreover, the federated Q-value prediction model is trained for 7200 episodes in three different areas. During training, iterate 25 steps per episode, and the federal aggregation K frequency is once every 10 episodes. The final model can adapt to dynamic scenarios of different area sizes. In the runtime environment, the threshold τ is 0.03, and it is considered that the agent has found a local task offloading solution when the Q-values are less than τ .

¹<http://sguangwang.com/TelecomDataset.html>²<https://cse.hkust.edu.hk/scrg/>

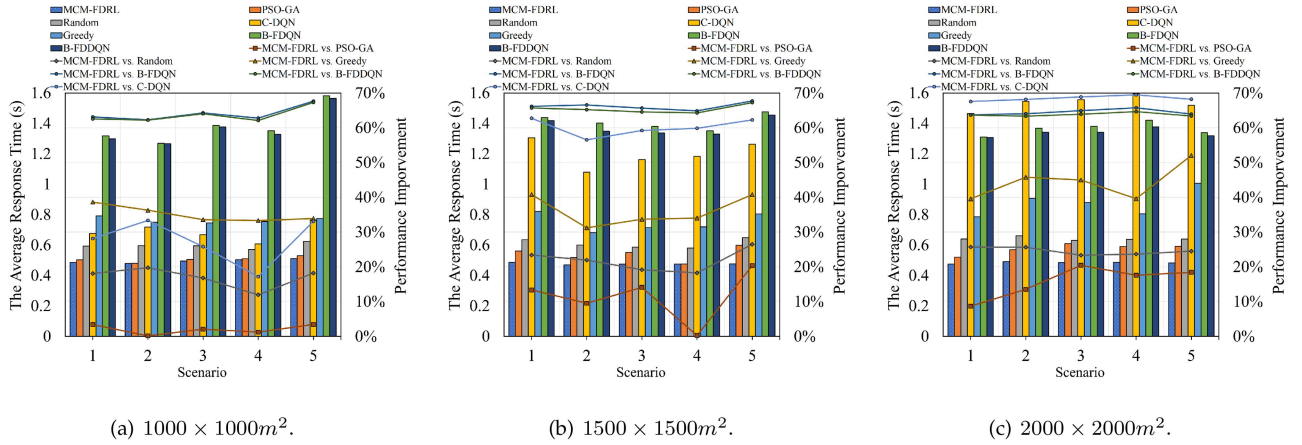


Fig. 5. Performance comparison between the MCM-FDRL and the state-of-the-art methods for various scenarios with different area sizes.

To verify the advantage of the MCM-FDRL for vehicle task offloading optimization, we make the comparison with the state-of-the-art methods as follows:

- Binary-FDQN (*B-FDQN*) [10]: In this method, vehicle tasks can be binary offloaded to one edge or executed locally by combining DQN and federated learning.
- Binary-FDDQN (*B-FDDQN*) [20]: In this method, vehicle tasks can be binary offloaded to one edge or executed locally by combining DDQN and federated learning.
- Particle Swarm Optimization and Genetic Algorithm (*PSO-GA*) [8]: This method adopts crossover and mutation operators of GA to improve the particle update of the classical PSO. Specifically, each particle represents a task offloading plan for all vehicles, and each quantize of the particle is encoded as the offloading location and the data volume offloaded to the corresponding vehicle location. The population size, cognitive factor c_1 , c_2 , and inertia weight factor w are set to 100, 0.5, 0.5, and 0.8, respectively.
- Centralized-DQN (*C-DQN*) [41]: This method ignores the density of vehicles and takes the remaining vehicle information as a whole. The DQN takes the whole information of vehicles as an input for centralized decision making.
- *Greedy*: This method guides each vehicle in selecting the actions that can minimize the average response time according to all vehicles' current task offloading plans. Then, the offloading decisions of vehicles are continuously updated one by one.
- Random Search (*Random*): This method allows all vehicles to select their task offloading plans randomly, and then the optimal result is selected as the final task offloading plan after 100000 repeated experiments.

B. RQ 1: Performance Improvement of the MCM-FDRL Compared to the State-of-the-Art Methods

Fig. 5 shows the performance comparison between the MCM-FDRL and the state-of-the-art methods for various scenarios with different area sizes. The figure shows that the performance

TABLE V
EXECUTION TIME TO FIND OBJECTIVE VEHICULAR TASK OFFLOADING PLANS (s)

Area	MCM-FDRL	B-FDQN	B-FDDQN	C-DQN	PSO-GA	Greedy	Random
1	0.29	0.05	0.13	0.92	115.15	0.44	93.11
2	0.87	0.39	0.63	4.63	608.70	4.58	404.46
3	1.96	0.87	1.38	6.17	1826.20	22.05	1214.44

of MCM-FDRL is better than that of PSO-GA, C-DQN, Random, Greedy, B-FDQN, and B-FDDQN.

MCM-FDRL reduces the average task response time by 2.03% to 15.72%, 27.50% to 68.45%, 16.94% to 24.55%, and 35.13% to 44.38% compared to centralized methods such as PSO-GA, C-DQN, respectively. Specifically, PSO-GA tends to fall into local optimum and cannot find the optimal solution when facing a large solution space. C-DQN makes centralized decisions based on global information, the state space and action space become larger and cannot maintain stable performance when the size increases. Random uses a randomized approach without the guidance of a strategy. Greedy only considers the current optimal offloading solution and only obtains local optimum. MCM-FDRL reduces the average task response time by 64.02% to 66.19% and 63.63% to 65.40% compared to distributed methods such as B-FDQN and B-FDDQN, respectively. B-FDQN and B-FDDQN use binary offloading decision, i.e., the task is either offloaded or not offloaded, and do not consider the divisibility of the task, which limits the fineness of the decision and the offloading efficiency is not high. In contrast, MCM-FDRL employs distributed decision making and considers task divisibility, utilizing partial offloading decisions to improve the fineness of decision making.

As shown in Table V, we evaluate the execution time of different methods to find their objective vehicular task offloading plans. For each area size, the execution time of a method in five scenarios is averaged. The execution time of MCM-FDRL is reduced by more than 99% compared to both PSO-GA and Random. This is because PSO-GA and Random take a lot of time to search for solutions when faced with a large solution

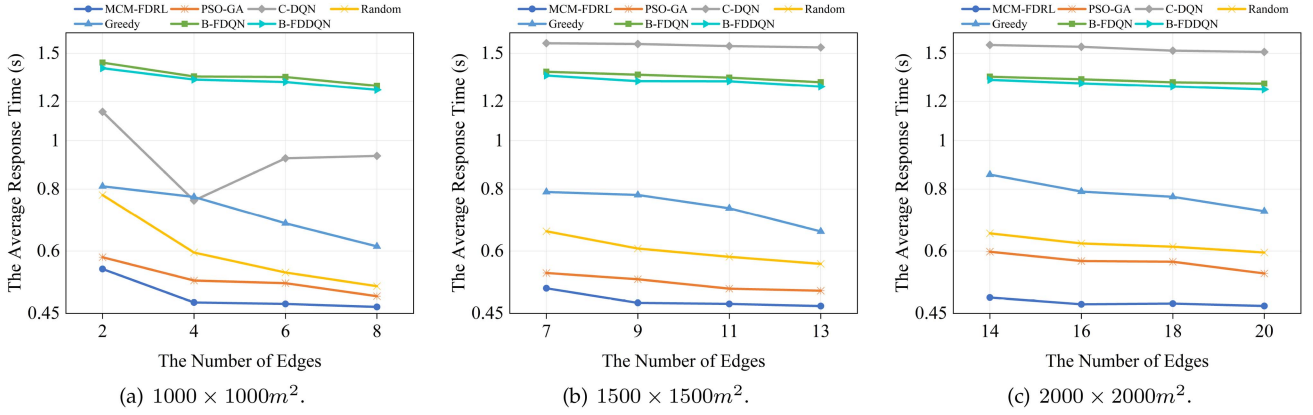


Fig. 6. Effect of various scales of edges on the average response time achieved by different methods.

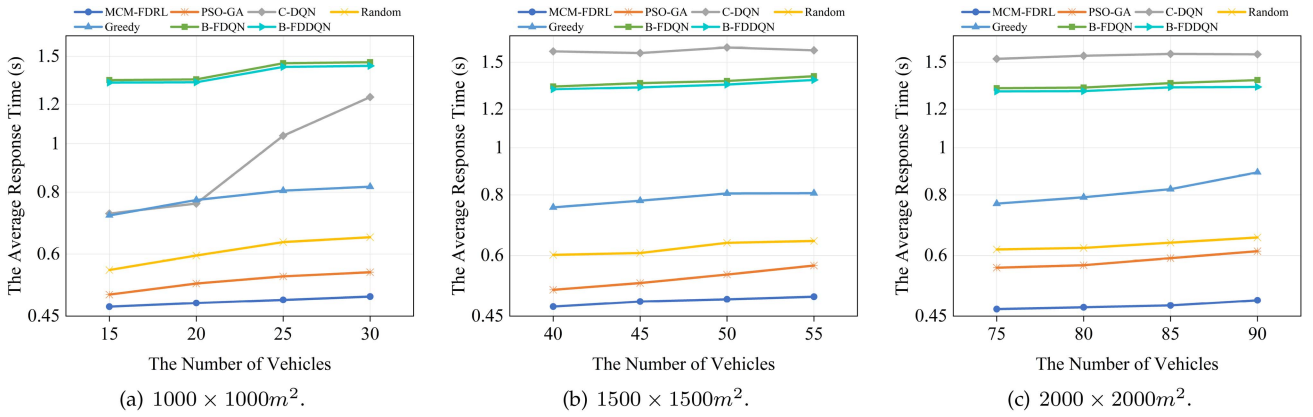


Fig. 7. Effect of various scales of vehicles on the average response time achieved by different methods.

space. The execution time of MCM-FDRL is in the same order of magnitude as that of C-DQN, Greedy, B-FDQN, and B-FDDQN. Among them, B-FDQN and B-FDDQN use binary offloading decisions, which have a smaller solution space, and thus have a shorter execution time than MCM-FDRL.

C. RQ 2: Effect of Various Scales of Edges and Vehicles on Performance of Different Methods

We test the performance of different methods with various scales of vehicles and edges. As shown in Fig. 6, we first test the effect of various scales of edges on the average response time achieved by different methods. When the number of vehicles M is constant, the average response time of each method decreases to different degrees as the number of edges increases. There are more edges for vehicles to choose to offload their tasks, and the distance between vehicles and edges also becomes closer so that the offloading time of the task is also correspondingly reduced. The average response time of the MCM-FDRL is better than that of PSO-GA, C-DQN, Greedy, Random, B-FDQN, and B-FDDQN. As the number of edges increases, the performance of centralized methods such as PSO-GA, C-DQN, Greedy, and Random gradually lag behind that of the proposed MCM-FDRL.

This is because centralized methods tend to fall into local optimums and fail to find the optimal solution when facing a large solution space. In contrast, the performance gap between MCM-FDRL and B-FDQN and B-FDDQN, which also use distributed methods, is essentially the same.

Next, we test the effect of various vehicle scales on the average response time achieved by different methods. As shown in Fig. 7, when the number of edges N is constant, the average response time of each method increases to different degrees as the number of vehicles increases. This phenomenon is because as the number of vehicles increases, the edge needs to provide more services to the vehicles. The limited computing resources of the edge lead to an increase in the load ratio of the edge, which leads to an increase in the average response latency of the edge-processing tasks, and the vehicles may offload to more distant edges, leading to an increase in the transmission latency. Besides, the vehicles may execute their tasks locally or offload them to remote edges for execution, causing an increase in transmission latency. As the number of vehicles increases, the performance of centralized methods such as PSO-GA, C-DQN, Greedy, and Random gradually lag behind that of the proposed MCM-FDRL, which indicates that large solution space dramatically limits the performance of centralized methods. In contrast,

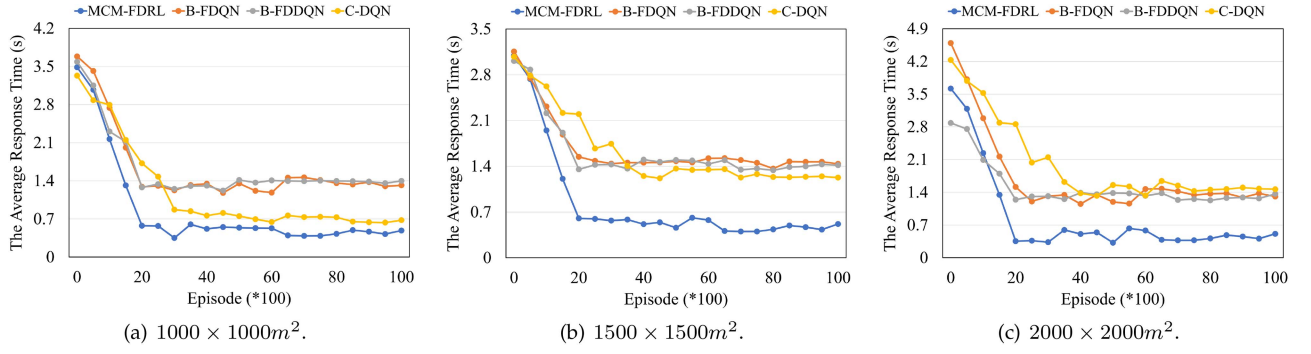


Fig. 8. Convergence performance of the MCM-FDRL with different methods.

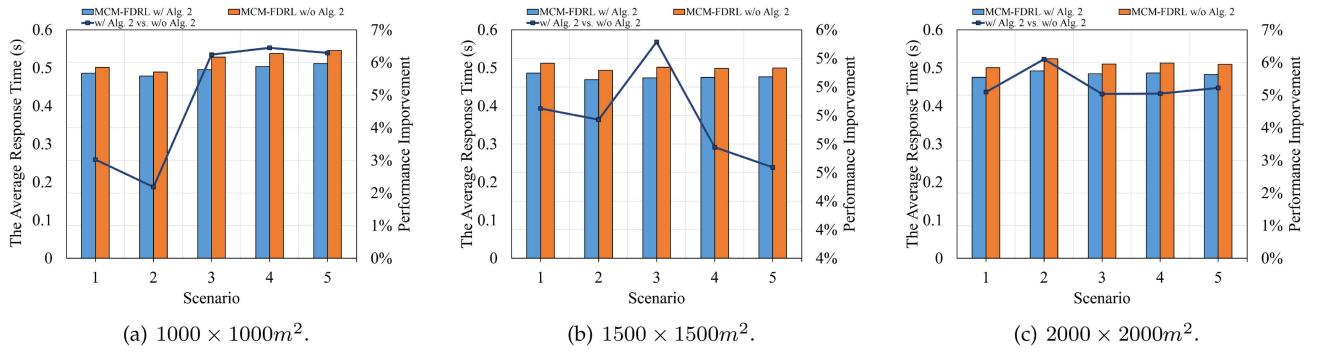


Fig. 9. Effect of using the density of vehicles at the edges as the state on the average response time achieved by the MCM-FDRL.

the performance gap between MCM-FDRL and B-FDQN and B-FDDQN, which also use distributed methods, is essentially the same.

D. RQ 3: Convergence Performance of the MCM-FDRL

We test the convergence performance of MCM-FDRL and other DRL-based methods in different scenarios. As shown in Fig. 8, we test the influence of different numbers of training episodes on the average response time. After about 2000 training episodes, MCM-FDRL outperforms other methods. In particular, MCM-FDRL with B-FDQN and B-FDDQN that is trained by federated learning achieves better performance at 2000 training episodes, and the performance remains stable after around 6500 training episodes. In contrast, C-DQN uses centralized training and achieves good performance at 2500 training episodes, and the performance remains unchanged after around 7500 training episodes, indicating slower convergence than federated learning.

E. RQ 4: Performance Improvement Using Vehicle Density at Edges as State Inputs

Finally, to evaluate the effectiveness of using edge vehicle density as state input in the method (i.e., Algorithm 2), we test the performance improvement of the MCM-FDRL when using Algorithm 2. As shown in Fig. 9, we compare the average response time for scenarios with/without Algorithm 2.

Specifically, the average task response time obtained by MCM-FDRL are reduced by 5.04% on average compared to without Algorithm 2. This not only reflects the performance improvement of MCM-FDRL in practical applications, but also reveals the advantages behind it. By using the density of vehicles at the edges as state input, MCM-FDRL is able to sense global information more comprehensively, which means that the system is able to more accurately understand the current network conditions, vehicle distribution, and task requirements. Based on this comprehensive information, MCM-FDRL is able to develop a finer and more reasonable task offloading plan. Such a plan can not only improve the processing efficiency of tasks and reduce waiting time, but also optimize the use of resources to ensure that the system maintains efficient and stable operation in a complex and changing environment. Therefore, by taking the density of vehicles at the edges into consideration, MCM-FDRL successfully realizes the accurate perception of global information and formulates a more appropriate task offloading plan accordingly, which significantly improves the performance of the system.

Moreover, we compare the time cost of finding objective vehicle task offloading plans, as shown in Table VI. When Algorithm 2 is adopted, the execution time of the MCM-FDRL is reduced by 30.46% on average. This is because the proposed MCM-FDRL takes the density of vehicles at the edges as the input for the vehicle, allowing the vehicle to perceive the future loads of different edges based on the density of vehicles at different edges. Compared to the MCM-FDRL without Algorithm 2,

TABLE VI
EXECUTION TIME TO FIND OBJECTIVE VEHICULAR TASK OFFLOADING PLANS (s)

Area	MCM-FDRL w/ Alg. 2	MCM-FDRL w/o Alg. 2
1	0.29	0.50
2	0.87	1.39
3	1.96	2.43

it can avoid unnecessary repetitive adjustment actions and only requires fewer feedback iterations to search for optimal plans. Therefore, using the density of vehicles at the edges as a state input enables the MCM-FDRL to achieve better vehicle task offloading plans in a shorter time.

VI. DISCUSSION

In our evaluation, we establish a VEC environment using real datasets with longitude and latitude information from Shanghai. We consider 15 scenarios varying in area size, different number of edges, and different number of vehicles to maximally simulate real-world environments. However, several issues still need to be addressed for practical application.

First, we simplify the communication model and computation model, where the network conditions between vehicles and edges at the same location are generally close and are assumed not to be interfered, and the compatibility of different computational resources with different tasks is similar. It is noted that this paper focuses on dynamic task offloading in VEC, which is orthogonal to the above factors. To better apply our work to practical scenarios, we will further consider the above factors by refining the communication model by introducing Gaussian white noise [42], Rayleigh fading channels [43], signal-to-interference-to-noise ratio [44], shadow fading [45], and support complex computation models by considering heterogeneous edge servers [20].

Second, since the connection between vehicles and edges and the load of edges are constantly changing in real-world environments, to simplify the dynamics of VEC environments, we consider the connection and load to be constant during the decision-making process and ignore the communication overhead of obtaining the above information. In fact, each vehicle makes decisions independently based on local information, and it only needs to connect several edges to obtain the edge information. Therefore, compared to the centralized decision-making manner, our proposed method is much less affected by the dynamic changes of VEC environments. In future work, we will further consider the communication overhead and its impact on the decision-making process [46], [47] and improve the extensibility of our method in real-world environments.

Third, online federated reinforcement learning causes high overheads in practical environments, but we adopt an offline framework to the training model, where federated reinforcement learning is employed to accelerate the training process. Therefore, we ignore the extra overheads of federated reinforcement learning in practical deployment. In future work, we will further explore more advanced federated reinforcement learning

models [22], [48] to solve the practical deployment problem in resource-constrained environments through lightweight design and pruning.

VII. CONCLUSION

In this paper, we propose the MCM-FDRL to address the large-scale vehicular task offloading problem in dynamic VEC systems. First, we formulate each vehicle as a DRL agent that can independently make offloading decisions based on local information. Next, to obtain the global information without adding too much overhead, we introduce the density of vehicles at the edges into the local information of the vehicle agent, which avoids vehicles offloading tasks to the edge with higher loads. Finally, we use the MCM-FDRL method to train the Q-value prediction model for task offloading and find an effective vehicle task offloading plan through multi-agent collaboration. Extensive simulation results verify good adaptability of the proposed MCM-FDRL, which outperforms the PSO-GA, C-DQN, Random, Greedy, B-FDQN, and B-FDDQN by about 9.75%, 52.06%, 21.11%, 38.53%, 64.90%, and 64.27% in terms of the average response time for different scenarios, respectively. Meanwhile, the global Q-value prediction model trained through federated learning is extensible and can be applied to dynamic environments with different numbers of vehicles and edges.

REFERENCES

- [1] J. Feng, Z. Liu, C. Wu, and Y. Ji, "Mobile edge computing for the Internet of Vehicles: Offloading framework and job scheduling," *IEEE Veh. Technol. Mag.*, vol. 14, no. 1, pp. 28–36, Mar. 2019.
- [2] W. Zhan et al., "Deep-reinforcement-learning-based offloading scheduling for vehicular edge computing," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5449–5465, Jun. 2020.
- [3] S. Tuli, S. Ilager, K. Ramamohanarao, and R. Buyya, "Dynamic scheduling for stochastic edge-cloud computing environments using A3C learning and residual recurrent neural networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 3, pp. 940–954, Mar. 2022.
- [4] Y. Wang, I.-R. Chen, and D.-C. Wang, "A survey of mobile cloud computing applications: Perspectives and challenges," *Wireless Pers. Commun.*, vol. 80, pp. 1607–1623, 2015.
- [5] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surv. Tut.*, vol. 19, no. 3, pp. 1628–1656, Third Quarter 2017.
- [6] Y. Peng et al., "Computing and communication cost-aware service migration enabled by transfer reinforcement learning for dynamic vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 257–269, Jan. 2024.
- [7] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint load balancing and offloading in vehicular edge computing and networks," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4377–4387, Jun. 2019.
- [8] Q. Luo, C. Li, T. H. Luan, and W. Shi, "Minimizing the delay and cost of computation offloading for vehicular edge computing," *IEEE Trans. Serv. Comput.*, vol. 15, no. 5, pp. 2897–2909, Sep./Oct. 2022.
- [9] W. Lv et al., "Energy consumption and QoS-aware co-offloading for vehicular edge computing," *IEEE Internet Things J.*, vol. 10, no. 6, pp. 5214–5225, Mar. 2023.
- [10] Q. Zhang, H. Wen, Y. Liu, S. Chang, and Z. Han, "Federated-reinforcement-learning-enabled joint communication, sensing, and computing resources allocation in connected automated vehicles networks," *IEEE Internet Things J.*, vol. 9, no. 22, pp. 23224–23240, Nov. 2022.
- [11] M. Tiwari, I. Maity, and S. Misra, "FedServ: Federated task service in fog-enabled Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 20943–20952, Nov. 2022.

- [12] F. Jiang, W. Liu, J. Wang, and X. Liu, "Q-learning based task offloading and resource allocation scheme for Internet of Vehicles," in *Proc. 2020 IEEE/CIC Int. Conf. Commun. China*, 2020, pp. 460–465.
- [13] K. Zhang, Y. Zhu, S. Leng, Y. He, S. Maharjan, and Y. Zhang, "Deep learning empowered task offloading for mobile edge computing in urban informatics," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 7635–7647, Oct. 2019.
- [14] K. Zhang, S. Leng, X. Peng, L. Pan, S. Maharjan, and Y. Zhang, "Artificial intelligence inspired transmission scheduling in cognitive vehicular communications and networks," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 1987–1997, Apr. 2019.
- [15] Z. Xiao, J. Shu, H. Jiang, G. Min, H. Chen, and Z. Han, "Perception task offloading with collaborative computation for autonomous driving," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 2, pp. 457–473, Feb. 2023.
- [16] Q. Luo, C. Li, T. H. Luan, W. Shi, and W. Wu, "Self-learning based computation offloading for Internet of Vehicles: Model and algorithm," *IEEE Trans. Wireless Commun.*, vol. 20, no. 9, pp. 5913–5925, Sep. 2021.
- [17] X. Xu, Z. Fang, L. Qi, W. Dou, Q. He, and Y. Duan, "A deep reinforcement learning-based distributed service offloading method for edge computing empowered Internet of Vehicles," *Jisuanji Xuebao/Chinese J. Comput.*, vol. 44, no. 12, pp. 2382–2405, 2021.
- [18] X. Zhu, Y. Luo, A. Liu, M. Z. A. Bhuiyan, and S. Zhang, "Multiagent deep reinforcement learning for vehicular computation offloading in IoT," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9763–9773, 2021.
- [19] G. Fragkos, N. Kemp, E. E. Tsiropoulou, and S. Papavassiliou, "Artificial intelligence empowered UAVs data offloading in mobile edge computing," in *Proc. IEEE Int. Conf. Commun.*, 2020, pp. 1–7.
- [20] J. Li, Z. Yang, X. Wang, Y. Xia, and S. Ni, "Task offloading mechanism based on federated reinforcement learning in mobile edge computing," *Digit. Commun. Netw.*, vol. 9, no. 2, pp. 492–504, 2023.
- [21] Z. Tong, J. Wang, J. Mei, K. Li, W. Li, and K. Li, "Multi-type task offloading for wireless Internet of Things by federated deep reinforcement learning," *Future Gener. Comput. Syst.*, vol. 145, pp. 536–549, 2023.
- [22] X. Chen et al., "Traffic prediction-assisted federated deep reinforcement learning for service migration in digital twins-enabled MEC networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 10, pp. 3212–3229, Oct. 2023.
- [23] H. Zhou, H. Wang, Z. Yu, G. Bin, M. Xiao, and J. Wu, "Federated distributed deep reinforcement learning for recommendation-enabled edge caching," *IEEE Trans. Services Comput.*, vol. 17, no. 6, pp. 3640–3656, Nov./Dec. 2024.
- [24] H. Ke, J. Wang, L. Deng, Y. Ge, and H. Wang, "Deep reinforcement learning-based adaptive computation offloading for mec in heterogeneous vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7916–7929, Jul. 2020.
- [25] L. Kleinrock, Theory, *Queueing Systems*, vol. 1. Hoboken, NJ, USA: Wiley, 1975.
- [26] Z. Liu, P. Dai, H. Xing, Z. Yu, and W. Zhang, "A distributed algorithm for task offloading in vehicular networks with hybrid fog/cloud computing," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 52, no. 7, pp. 4388–4401, Jul. 2022.
- [27] C. J. C. H. Watkins, "Learning from delayed rewards," PhD thesis, King's College, Oxford, 1989.
- [28] T. Li, S. Ying, Y. Zhao, and J. Shang, "Batch jobs load balancing scheduling in cloud computing using distributional reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 1, pp. 169–185, Jan. 2024.
- [29] K. R. Alasmari, R. C. G. II, and M. Alam, "Mobile edge offloading using Markov decision processes," in *Proc. Edge Comput. - EDGE 2018*, Seattle, WA, USA, 2018.
- [30] Y. Zhou, Q. Ye, and J. Lv, "Communication-efficient federated learning with compensated overlap-fedavg," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 1, pp. 192–205, Jan. 2022.
- [31] Y. Li, A. Zhou, X. Ma, and S. Wang, "Profit-aware edge server placement," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 55–67, Jan. 2022.
- [32] Y. Guo, S. Wang, A. Zhou, J. Xu, J. Yuan, and C.-H. Hsu, "User allocation-aware edge cloud placement in mobile edge computing," *Softw.: Pract. Experience*, vol. 50, no. 5, pp. 489–502, 2020.
- [33] S. Wang, Y. Guo, N. Zhang, P. Yang, A. Zhou, and X. Shen, "Delay-aware microservice coordination in mobile edge computing: A reinforcement learning approach," *IEEE Trans. Mobile Comput.*, vol. 20, no. 3, pp. 939–951, Mar. 2021.
- [34] W. Feng, S. Lin, N. Zhang, G. Wang, B. Ai, and L. Cai, "Joint C-V2X based offloading and resource allocation in multi-tier vehicular edge computing system," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 2, pp. 432–445, Feb. 2023.
- [35] Y. Wang et al., "A game-based computation offloading method in vehicular multiaccess edge computing networks," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 4987–4996, Jun. 2020.
- [36] Q. Luo, C. Li, T. H. Luan, and W. Shi, "Collaborative data scheduling for vehicular edge computing via deep reinforcement learning," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9637–9650, Oct. 2020.
- [37] X. Chen, Z. Yao, Z. Chen, G. Min, X. Zheng, and C. Rong, "Load balancing for multi-edge collaboration in wireless metropolitan area networks: A two-stage decision-making approach," *IEEE Internet Things J.*, vol. 10, no. 19, pp. 17124–17136, Oct. 2023.
- [38] M. Abadi et al., "Tensorflow: A system for large-scale machine learning," in *Proc. 12th USENIX Conf. Operating Syst. Des. Implementation*, USENIX Association, 2016, pp. 265–283.
- [39] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27th Int. Conf. Mach. Learn.*, Haifa, Israel, 2010, pp. 807–814.
- [40] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations*, San Diego, CA, USA, 2015, pp. 1–15.
- [41] Q. Liu, L. Shi, L. Sun, J. Li, M. Ding, and F. Shu, "Path planning for UAV-mounted mobile edge computing with deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 69, no. 5, pp. 5723–5728, May 2020.
- [42] L. Wan, L. Sun, K. Liu, X. Wang, Q. Lin, and T. Zhu, "Autonomous vehicle source enumeration exploiting non-cooperative UAV in software defined Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 6, pp. 3603–3615, Jun. 2021.
- [43] A. Pandey, S. Yadav, D.-T. Do, and R. Kharel, "Secrecy performance of cooperative cognitive AF relaying networks with direct links over mixed Rayleigh and double-Rayleigh fading channels," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 15095–15112, Dec. 2020.
- [44] X. Ma and K. S. Trivedi, "SINR-based analysis of IEEE 802.11p/bd broadcast VANETs for safety services," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 3, pp. 2672–2686, Sep. 2021.
- [45] N. E.-D. Safwat, R. Sabatini, A. Gardi, I. M. Hafez, and F. Newagy, "Urban air mobility communication performance considering cochannel interference," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 60, no. 4, pp. 5089–5100, Aug. 2024.
- [46] L. Liu, M. Zhao, M. Yu, M. A. Jan, D. Lan, and A. Taherkordi, "Mobility-aware multi-hop task offloading for autonomous driving in vehicular edge computing and networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2169–2182, Feb. 2023.
- [47] Z. Gao, L. Yang, and Y. Dai, "Fast adaptive task offloading and resource allocation in large-scale MEC systems via multiagent graph reinforcement learning," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 758–776, Jan. 2024.
- [48] H. Zhou, M. Li, P. Sun, B. Guo, and Z. Yu, "Accelerating federated learning via parameter selection and pre-synchronization in mobile edge-cloud networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 11, pp. 10313–10328, Nov. 2024.



IEEE Transactions on Industrial Informatics, etc.



Xing Chen (Member, IEEE) is a professor with Fuzhou University, and the director of Fujian Key Laboratory of Network Computing and Intelligent Information Processing. He focuses on the software systems and engineering approaches for cloud and mobility. His current projects cover the topics from self-adaptive software, computation offloading, model driven approach and so on. He has published more than 80 journal and conference articles, including *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Cloud Computing*,

Bohuai Xiao received the MS degree in the Guilin University of Technology, China, in 2023. He is currently working toward the PhD degree in computer technology in the College of Computer and Data Science, Fuzhou University. His current research interests include cloud/edge computing, computation offloading, graph neural network, and reinforcement learning.



Xinyu Lin received the BE degree in computer science and technology from Huaqiao University, China, in 2021. She is currently working toward the MS degree in computer applied technology in the College of Computer and Data Science, Fuzhou University, China. Her current research interests include cloud-edge computing, computation offloading, and deep reinforcement learning.



Geyong Min (Member, IEEE) received the BSc degree in computer science from the Huazhong University of Science and Technology, China, in 1995 and the PhD degree in computing science from the University of Glasgow, United Kingdom, in 2003. He is a professor of High Performance Computing and Networking in the Department of Computer Science, University of Exeter, United Kingdom. His research interests include future Internet, computer networks, wireless communications, multimedia systems, information security, high-performance computing, ubiquitous computing, modelling, and performance engineering.



Zheyi Chen (Member, IEEE) received the PhD degree in computer science from the University of Exeter, U.K., in 2021, and the MSc degree in computer science and technology from Tsinghua University, China, in 2017, respectively. He is a professor and Qishan Scholar with the College of Computer and Data Science with the Fuzhou University, China. His research interests include cloud-edge computing, resource optimization, deep learning, and reinforcement learning. Dr. Chen has published more than 30 research papers in reputable international journals

and conferences, such as *IEEE Transactions on Parallel and Distributed Systems*, *IEEE INFOCOM*, *IEEE Transactions on Industrial Informatics*, *IEEE Communications Magazine*, *IEEE Transactions on Cloud Computing*, *IEEE IoT Journal*, and *IEEE ICC*.